

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



Kollektive Intelligenz und Schwarmverhalten

„Das Ganze ist mehr als die Summe seiner Teile.“
(Paul Klee)

PD Prof. Dr. Marco Block-Berlitz

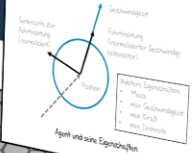
Schwarmsimulation

Hier sehen wir 10 Agenten in Aktion. Die beiden relevanten Vektoren einwirkende Steuerungskraft (Beschleunigung) und Bewegungsrichtung werden angezeigt.



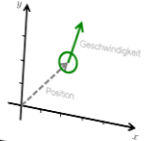
Überlegungen zu einem Agentenmodell

Ein Agent sollte mindestens seine **Position** im Raum und seine **Bewegungsrichtung** kennen. Um Objekte im 3D-Raum korrekt zu visualisieren, benötigen wir drei Achsen. Die **Bewegungsrichtung** könnte eine sein, wenn wir die senkrechte Richtung dazu kennen, können wir über das Kreuzprodukt die dritte Achse ermitteln. Weiterhin beeinflussen Objekteigenschaften wie **Mass**, **maximale Geschwindigkeit**, **maximale Beschleunigung**, **Drehrate** usw. die Bewegung eines Objektes im Raum.



Agent und seine Eigenschaften

- max. Geschwindigkeit
- max. Beschleunigung
- max. Kraft
- max. Drehrate



51

Ziele und Inhalt

- Von der Spielumgebung zum Schwarm
- Viele Objekte sind noch kein Schwarm
- Verhalten kapseln war der erste Schritt
- Klassen Vektor2D, Vektor3D und LineareAlgebra
- Klassen auf Vektoren umstellen
- Ein Agent ist mehr als ein bewegliches Objekt
- Überlegungen zum Agentenmodell
- Agent als Erweiterung des Objektmodells
- Bewegung entsteht durch Kräfte
- Trennung von update und render
- Variante: Fixed-Time-Step
- Variante: Fixed-Update-Time-Step
- Vergleich mit und ohne dt
- Beschleunigung ins Spiel bringen
- Gesetze von Newton, Einführung der Masse
- Gravitationskraft, Reibungskraft
- Viskositäten von Flüssigkeiten, Federkraft
- Steuerungskraft: Ziel folgen (seek)
- Steuerungskraft: Fliehen (flee)
- Steuerungskraft: Hindernisumgehung (obstacle avoidance)
- Kollektive Intelligenz
- Flocking-Boids (Schwarmverhalten)
- Separation, Alignment, Cohesion
- Schwarmsimulationen, Tools in POGL
- Empfehlung 3Blue1Brown, Epidemic Simulation
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben

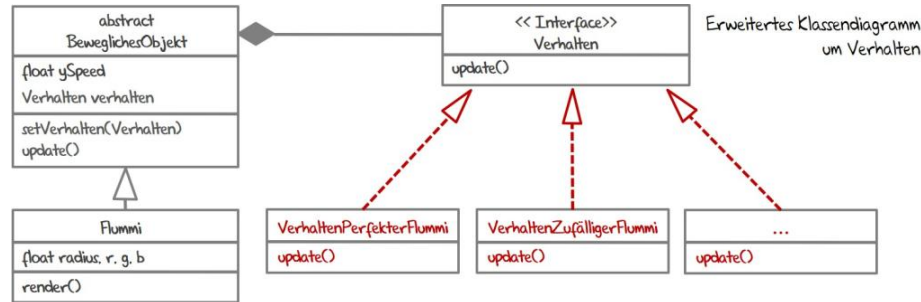




Rückblick auf die letzte Version der
Spiele- und Simulationsumgebung

Von der Spielumgebung zum Schwarm

Im letzten Teil haben wir eine kleine [Spielumgebung als Untersuchungslabor](#) aufgebaut. Wir können Objekte erzeugen, verwalten, visualisieren und ihnen Verhalten zuweisen. Heute erweitern wir dieses System zu einer physikbasierten Simulationsumgebung, in der viele einfache Agenten lokal interagieren und dadurch kollektives Verhalten erzeugen.



Diese Struktur ist bereits mehr als eine Spielumgebung: Sie ist ein [kleines Framework](#) für visuelle Simulationen. Gerade weil viele Objekte verwaltet, aktualisiert und dargestellt werden können, eignet sich die Architektur auch als Einstieg in [Partikelsysteme](#) und [Schwarmverhalten](#).



Viele Objekte sind noch kein Schwarm

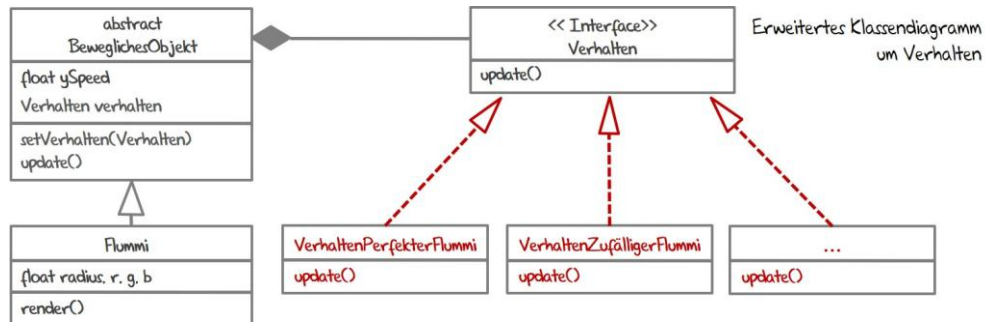
Im bisherigen System können wir bereits viele Flummies mit unterschiedlichem Verhalten verwalten. Diese Objekte verhalten sich jedoch weitgehend **isoliert**. Es fehlen: **lokale Wahrnehmung**, **physikbasierte Bewegungen**, **wechselseitige Beeinflussungen** und **emergente Gruppenmuster**. „Emergenz“ bedeutet hier, dass neue Verhaltensweisen erst durch das Zusammenspiel einzelner Gruppenmitglieder auftreten.



Kapselung von Verhalten war der erste Architekturbaustein

Die Kapselung von Verhalten über das [Strategy-Pattern](#) war im ersten Teil der entscheidende Architekturzug. Ein Objekt muss nicht seine Logik fest eingebaut besitzen. Stattdessen kann Verhalten ausgetauscht, erweitert und kombiniert werden. Genau diese Entkopplung macht es möglich, das System nun um [Steering](#) und [Schwarmregeln](#) zu erweitern.

```
public abstract class BeweglichesObjekt extends BasisObjekt {  
    public float ySpeed;  
    public Verhalten verhalten = null;  
  
    public BeweglichesObjekt(float xPos, float yPos, float ySpeed) {  
        super(xPos, yPos);  
        this.ySpeed = ySpeed;  
    }  
  
    public void setVerhalten(Verhalten verhalten) {  
        this.verhalten = verhalten;  
    }  
  
    public void update() {  
        if (verhalten != null)  
            verhalten.update();  
    }  
}
```





Wir überführen zunächst die Verwaltung der Positions- und Geschwindigkeitsparameter in **Vektoren**



Wir erinnern uns dazu an die Klassen, die wir bei der
Vektorengeometrie kennengelernt haben

Die Klassen Vektor2D, Vektor3D und LineareAlgebra

Kapitel 4 des Buches könnt Ihr Euch auf der vividus-Webseite herunterladen und direkt reinschauen [1]. Die Inhalte werden später in Beleuchtung und Rendering besprochen, aber die drei Klassen zur Linearen Algebra **Vektor2D**, **Vektor3D** und **LineareAlgebra** können wir jetzt bereits intuitiv einsetzen.

```
public class Vektor2D {  
    public double x, y;  
  
    public Vektor2D(double x, double y) {  
        setX(x);  
        setY(y);  
    }  
  
    public void add(Vektor2D vec) {  
        x += vec.x;  
        y += vec.y;  
    }  
  
    public void mult(double s) {  
        x *= s;  
        y *= s;  
    }  
    ...  
}
```

```
public class LineareAlgebra {  
    private LineareAlgebra() {};  
  
    public static Vektor2D add(Vektor2D vec1, Vektor2D vec2) {  
        return new Vektor2D(vec1.x+vec2.x, vec1.y+vec2.y);  
    }  
  
    public static Vektor3D add(Vektor3D vec1, Vektor3D vec2) {  
        return new Vektor3D(vec1.x+vec2.x, vec1.y+vec2.y, vec1.z+vec2.z);  
    }  
  
    public static Vektor2D mult(double s, Vektor2D vec) {  
        return mult(vec, s);  
    }  
    ...  
}
```



[1] Block-Berlitz, Marco: "Warum sich der Dino furchtbar erschreckte - Lehrbuch zu Beleuchtung und Rendering mit Java, LWJGL, OpenGL, OpenCV und GLSL", 3. Auflage, ISBN: 978-3-9819362-9-2, vividus Wissenschaftsverlag, 2021

BasisObjekt und BewegendesObjekt anpassen

Die Klassen **BasisObjekt** und **BewegendesObjekt** verwalten die Position und die Geschwindigkeit jetzt über **Vektor2D**.

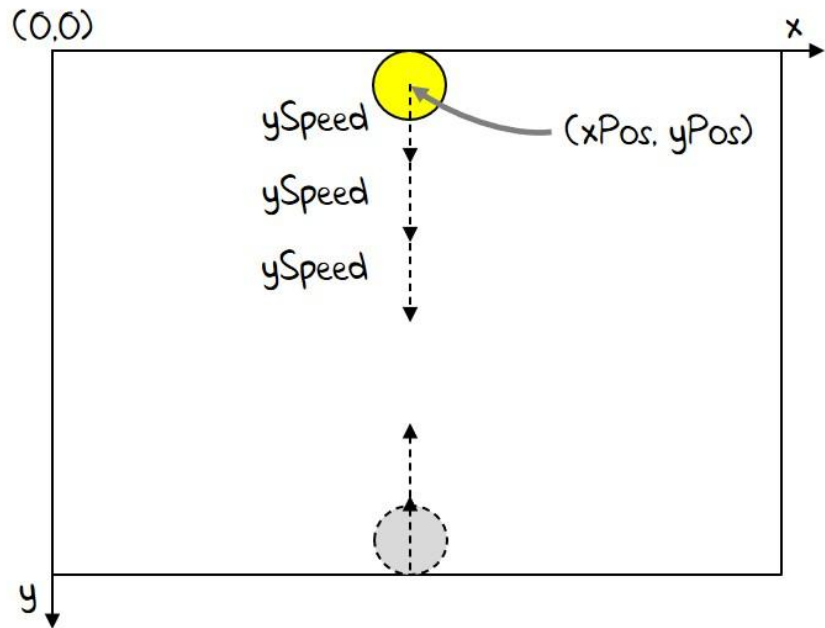
```
public abstract class BasisObjekt {  
    public int id;  
    public Vektor2D position;  
  
    public BasisObjekt() {  
        this(new Vektor2D(0,0));  
    }  
  
    public BasisObjekt(Vektor2D position) {  
        this.position = new Vektor2D(position);  
    }  
  
    public abstract void render();  
}
```

```
public abstract class BewegendesObjekt extends BasisObjekt {  
    public Vektor2D velocity;  
    public Verhalten verhalten = null;  
  
    public BewegendesObjekt(Vektor2D position, Vektor2D velocity) {  
        super(position);  
        this.velocity = new Vektor2D(velocity);  
    }  
  
    public void setVerhalten(Vehalten verhalten) {  
        this.verhalten = verhalten;  
    }  
  
    public void update() {  
        if (verhalten!=null)  
            verhalten.update();  
    }  
}
```

Verhalten anpassen

Auch die Manipulationen der Positionen und Geschwindigkeiten werden jetzt über die Klasse **Vektor2D** realisiert.

```
public class VerhaltenPerfekterFlummi implements Verhalten {  
    private Flummi flummi;  
  
    public VerhaltenPerfekterFlummi(Flummi flummi) {  
        this.flummi = flummi;  
    }  
  
    @Override  
    public void update() {  
        flummi.position.add(flummi.velocity);  
  
        if (flummi.position.y > 480 || flummi.position.y < 0)  
            flummi.velocity.y *= -1;  
    }  
}
```





Jetzt sind wir für den nächsten Schritt
gut vorbereitet!

Ein Agent ist mehr als ein bewegliches Objekt

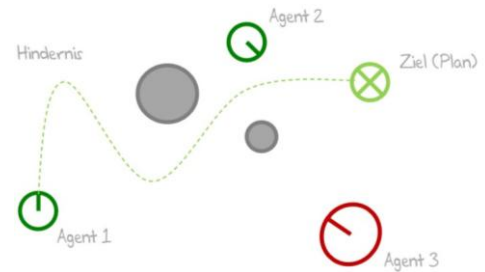
Ein autonomer Agent besitzt nicht nur eine **Position** und ein **Verhalten** (Zustand im Raum) Er benötigt weiterhin mindestens:

- eine **Bewegungsrichtung** bzw. **Geschwindigkeit**
- **Regeln zur Reaktion** auf seine Umgebung
- **Begrenzungen** wie maximale Geschwindigkeit oder maximale Kraft

Damit verschiebt sich der Fokus von „Was macht das Objekt?“ hin zu „Wie reagiert der Agent auf lokale Informationen der Umgebung?“

Ein autonomer Agent, bestimmt seine Bewegung innerhalb einer gegebenen Welt durch folgende drei Faktoren:

- Auswahl eines abstrakten Ziels/Plans (**Action selection**)
- Steuerung durch unterschiedlicher Kräfte, die physikalischer oder spieltechnischer Natur sind (**Steering**)
- Physikalisches Verhalten und Fähigkeiten der Fortbewegung (**Locomotion**)

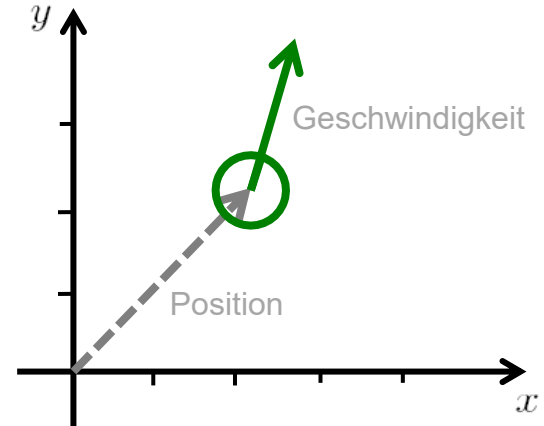
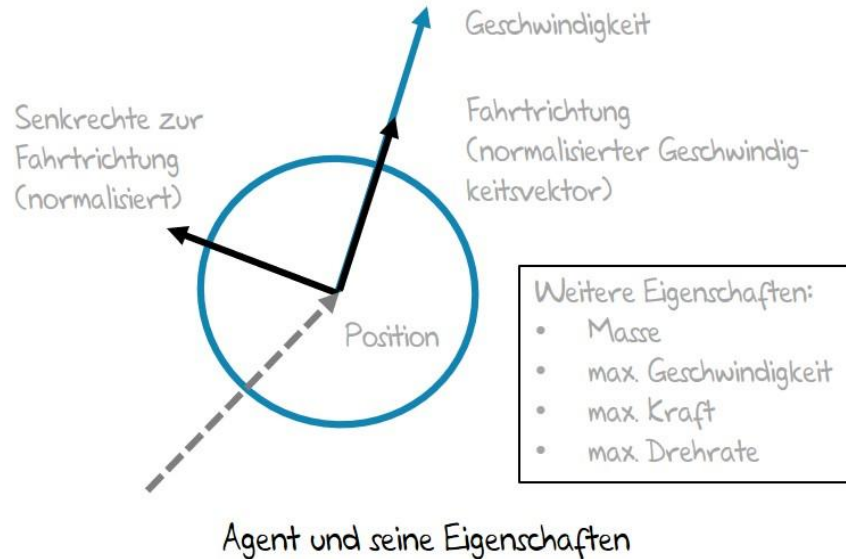


Steering - Behavior : nur auf Positionierung bezogen

Überlegungen zu einem Agentenmodell

Ein Agent sollte mindestens seine **Position** im Raum und seine **Bewegungsrichtung** kennen. Um Objekte im 3D-Raum korrekt zu visualisieren, benötigen wir drei Achsen. Die Bewegungsrichtung könnte eine sein, wenn wir die senkrechte Richtung dazu kennen, können wir über das Kreuzprodukt die dritte Achse ermitteln.

Weiterhin beeinflussen Objekteigenschaften, wie **Masse**, **maximale Geschwindigkeit**, **maximale Beschleunigung**, **Drehrate** usw. die Bewegung eines Objektes im Raum.



Agent als Erweiterung des Objektmodells

Um aus einem Flummi einen steuerbaren Agenten zu machen, benötigen wir zusätzlich zu Position und Geschwindigkeit eine Beschleunigung sowie obere Schranken für Kraft und Geschwindigkeit.

```
public abstract class Agent extends BeweglichesObjekt {  
    public Vektor2D acceleration;  
    public double maxSpeed = 4.0;  
    public double maxForce = 0.1;  
    public double mass = 1.0;  
  
    public void applyForce(Vektor2D force) {  
        Vektor2D scaled = LineareAlgebra.div(force, mass);  
        acceleration.add(scaled);  
    }  
  
    ...  
}
```

```
public void update() {  
    flummi.position.add(flummi.velocity);  
  
    if (flummi.position.y > 480 || flummi.position.y < 0)  
        flummi.velocity.y *= -1;  
}
```

Bisher wurde die Position direkt im Verhalten verändert. Für realistischere und kombinierbare Bewegungen führen wir jetzt **Kräfte** ein. Das Verhalten berechnet nicht mehr direkt die neue Position, sondern Kräfte, die auf das Objekt wirken.



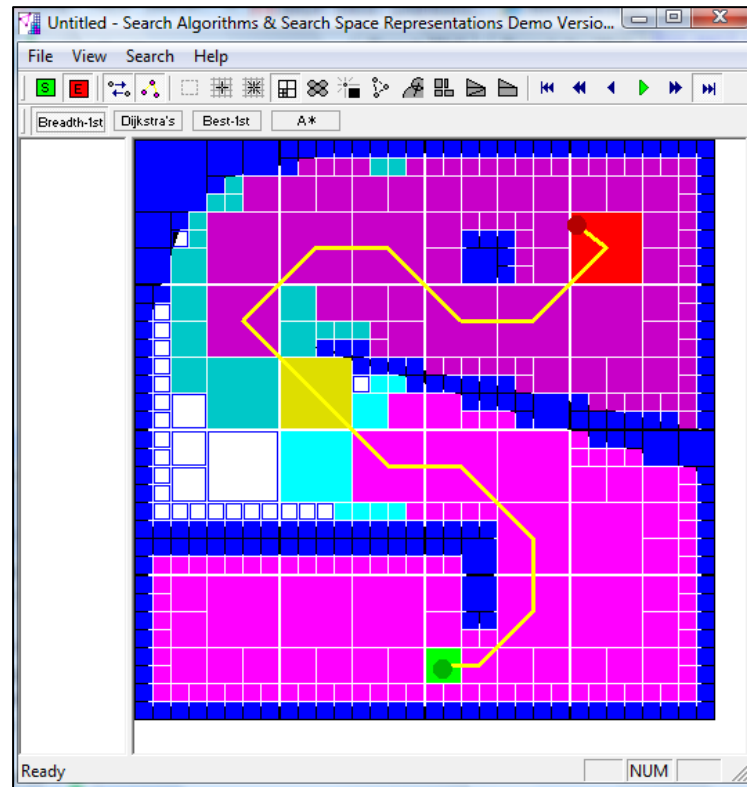
```
public void update() {  
    flummi.applyForce(separation(flummi, 20));  
    flummi.applyForce(alignment(flummi, 35));  
    flummi.applyForce(cohesion(flummi, 35));  
}
```

Bewegung entsteht durch Kräfte

Im Unterschied zu Suchraumrepräsentationen und Wegealgorithmen auf Graphen [1], ergibt sich die resultierende Beschleunigung aus der **gewichteten Summe aller Kräfte**.

Alle Bewegungen der Agenten werden im Folgenden als Kombination von Kräften modelliert:

- Gravitation
- Reibung
- Steuerungskräfte
- ...
- Schwarmregeln

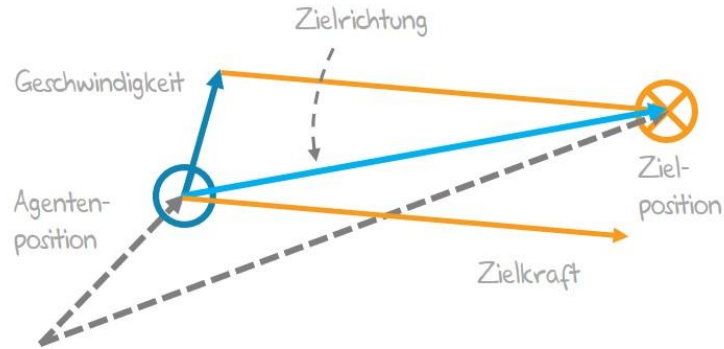




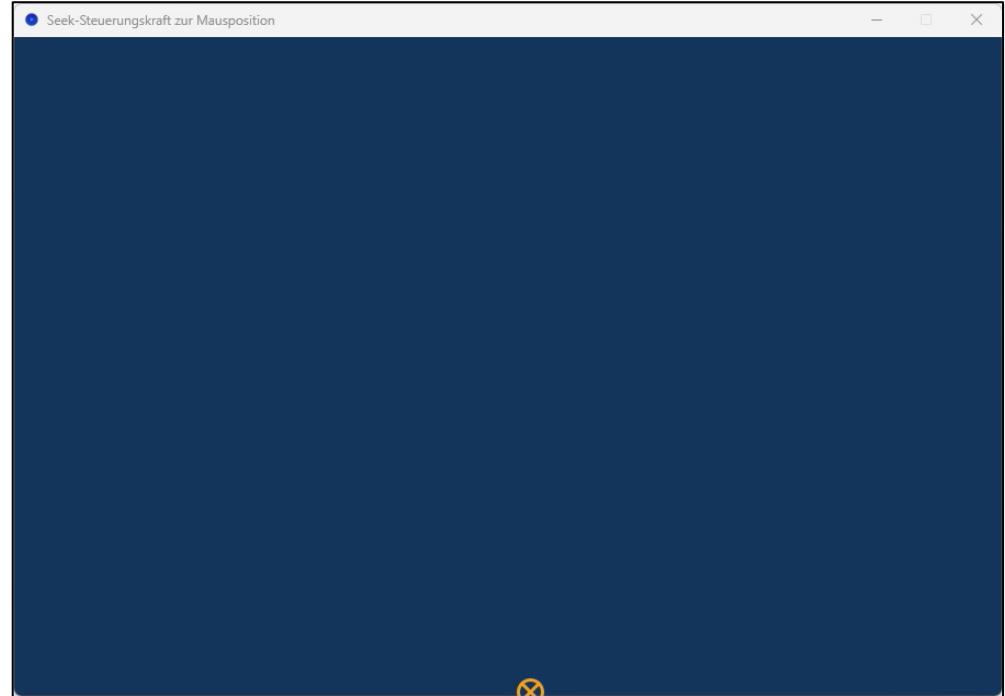
Schauen wir uns dazu mal ein Beispiel an

Steuerungskraft: Ziel folgen (seek)

Schauen wir uns die Wirkung einer Kraft auf einen Agenten am Beispiel „Ziel folgen“ an. Wenn wir einen Zielpunkt Ziel gegeben haben, dann gibt es eine Steuerungskraft, die direkt in die Richtung des Ziels zeigt.



Steuerungskraft „Ziel folgen“





Bevor es weiter geht, sollten wir die **Trennung** von **update** und **render** vornehmen

Trennung von update und render

Das bisher entwickelte System hatte **zwei große Nachteile**: Zum einen finden **update** und **render** ohne Synchronisierung **zum gleichen Zeitpunkt** statt und zum anderen sind unsere Objekte nur **mit konstanten Geschwindigkeiten** unterwegs. Beides ist sehr unschön und das wollen wir jetzt korrigieren.

```
@Override
public void renderLoop() {
    glEnable(GL_DEPTH_TEST);
    while (!Display.isCloseRequested()) {
        // ... hier OpenGL-Vorbereitungen für Hintergrund löschen usw.

        for (int i = 1; i <= flummies.getFlummiSize(); i++) {
            Flummi aktFlummi = flummies.getFlummi(i);
            aktFlummi.update();
            aktFlummi.render();
        }
        Display.update();
    }
    Display.destroy();
}
```

Wenn Simulation und Darstellung gekoppelt sind:

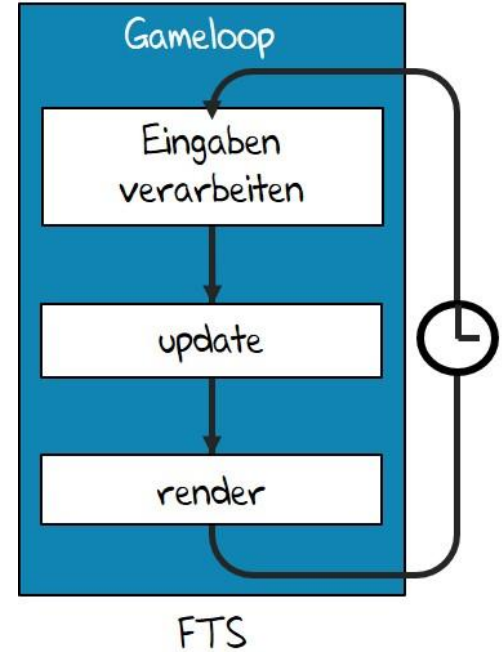
- hängt die Bewegung von der Bildrate ab
- wird die Simulation instabil
- sind Ergebnisse nicht reproduzierbar
- schneller Rechner → andere Bewegung
- langsamer Rechner → andere Bewegung

Beginnen wir mit der Frage, wie wir die **Unabhängigkeit** der Methoden **update** und **render** in den Griff bekommen. Da die Klärung dieser Frage genau so alt ist, wie die Computergrafik und speziell die Spieleentwicklung selbst, gibt es eine Reihe von Lösungsideen. Diese Ideen lassen sich auch als Design-Pattern abbilden.

Variante: Fixed-Time-Step

Eine Variante, die sich **Fixed-Time-Step** (FTS) mit Synchronisierung nennt, erzwingt einen gemeinsamen Durchlauf von möglichen Eingaben, **update** und **render** z.B. mit 60 FPS.

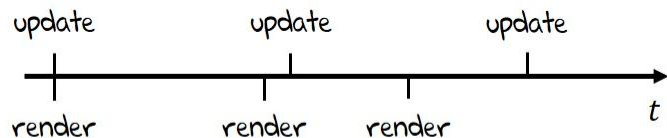
Dieses Konzept funktioniert, wenn das Programm viel zu schnell ist und wir damit eine konstante Bildfrequenz erzeugen können. Aber falls es zu langsam sein sollte, haben wir hier ein Problem.



Variante: Fixed-Update-Time-Step

Da das Rendern mal mehr oder weniger Zeit in Anspruch nimmt, die Physik bzw. KI aber einen relativ festen Takt braucht, werden wir in dieser Variante, dem **Fixed-Update-Time-Step** (FUTS) mit **variablem Rendering**, die **update**-Methode mit einem festen Zeitschritt aktualisieren und dem Rendern mehr Flexibilität zulassen.

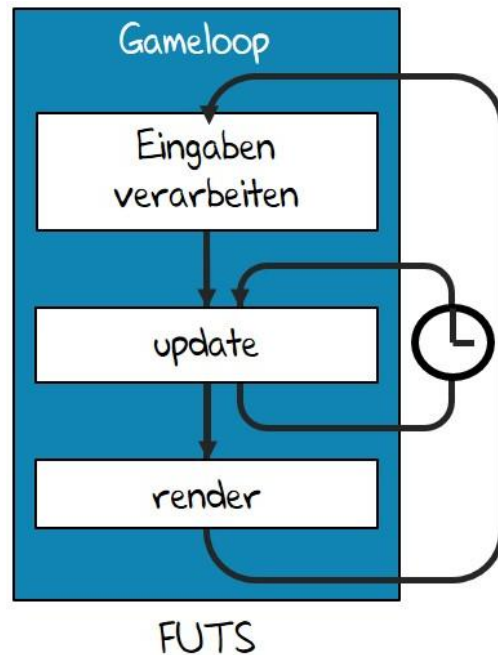
Es gibt allerdings noch ein Problem: Wenn wir die Physik nach einem festen Zeitschritt aktualisieren, aber das Rendern dann zu beliebigen Zeitpunkten stattfindet, kann es zu Zeitpunkten zwischen zwei Aktualisierungen kommen:



Eine Visualisierung von Positionen direkt zwischen zwei update-Schritten



Das können wir nur lösen, wenn wir die Methode **render** abhängig von **t** machen. Diese Extrapolation der Objekte ist eine Vermutung und kann auch manchmal falsch sein, ist aber in der Regel nicht bemerkbar.



Fixed-Update-Time-Step im Code

Schauen wir uns die letzte Variante kurz in einem Codeschnipsel an. Alle physikalischen Änderungen passieren hier in **update(dt)**. Das Rendering ist unabhängig davon.

```
double dt = 0.016;    // fester Simulationsschritt
double accumulator = 0.0;
long previousTime = System.nanoTime();

while (!Display.isCloseRequested()) {
    long currentTime = System.nanoTime();
    double frameTime = (currentTime - previousTime)/1_000_000_000.0;
    previousTime = currentTime;

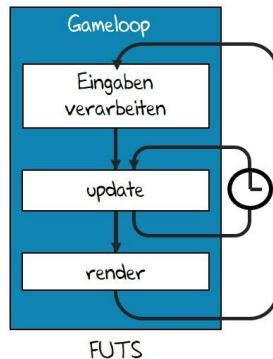
    // Schutz gegen extreme Ausreißer
    if (frameTime > 0.25) {
        frameTime = 0.25;
    }
    accumulator += frameTime;

    // Physik in festen Zeitschritten nachholen
    while (accumulator >= dt) {
        objektManager.updateAlle(dt);
        accumulator -= dt;
    }

    // Rendering unabhängig von der Zahl der Updates
    objektManager.renderAlle();
    Display.update();
}
```

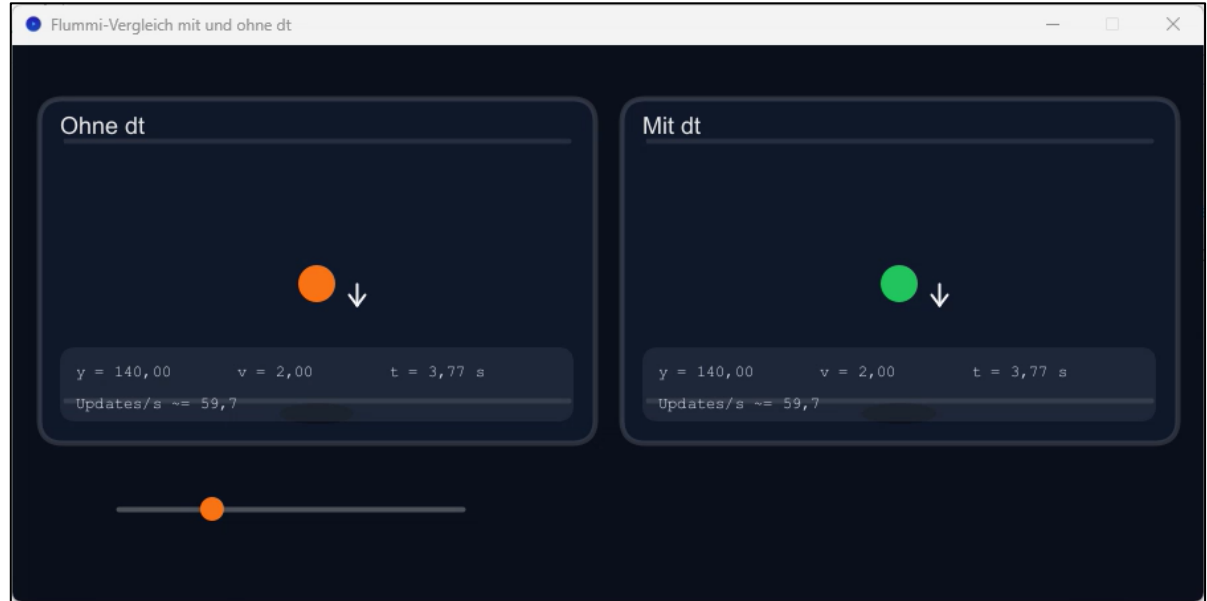
In der Praxis laufen Displays oft mit **60 Hz** (60 Updates pro Sekunde) was stabil für Simulationen ist. Daher setzen wir **double dt = 1.0/60;**

Damit erzeugen wir eine stabile und reproduzierbare Bewegung auf unterschiedlichen Rechnern.



Vergleich mit und ohne dt

Wir können die Verwendung von **dt** auf verschiedenen [Rechnersystemen simulieren](#). Links sehen wir den Einfluss der Rechnergeschwindigkeit auf die Physik, in diesem Fall den Flummi. Rechts sehen wir, dass die Physik unabhängig läuft, die Visualisierung macht bei einem sehr langsamen System zwar größere Sprünge, aber der [Flummi läuft konstant](#) weiter.





Jetzt müssen wir das Thema **physikalische Kräfte** erst einmal etwas mehr beleuchten ...

Beschleunigung ins Spiel bringen

Für ein [besseres Bewegungsmodell](#) erweitern wir das Objektmodell um die Variablen **position**, **velocity** und **acceleration** vom Typ **Vector3D**. Ab jetzt wollen wir uns um die Kräfte kümmern, die das Objekt beschleunigen und anschließend wie folgt die Geschwindigkeit und dann die Position des Objekts anpassen:

```
public void update() {  
    velocity.add(acceleration);  
    velocity.truncate(maxSpeed);  
    position.add(velocity);  
}
```

$$\begin{aligned}\vec{a} \\ \vec{v}_{t+1} &= \vec{v}_t + \vec{a} \\ \vec{p}_{t+1} &= \vec{p}_t + \vec{v}_{t+1}\end{aligned}$$

Diese einfache Möglichkeit, Kräfte in Positionsänderungen zu überführen, nennen wir [Euler-Integrationsmethode](#), explizites Euler-Verfahren oder auch Vorwärts-Euler-Verfahren. Ein weitaus besseres Verfahren wäre hier das Runge-Kutta-Verfahren, das z. B. in der Robotik häufig Verwendung findet.

Beschleunigung mit Integration der Zeit

Für die Integration in diskreten Zeitschritten erweitern wir die Euler-Integrationsmethode. Dabei werden Geschwindigkeit und Position in jedem Schritt über die seit dem letzten Schritt **vergangene Zeit Δt** fortgeschrieben [1, 2]:

```
public void update(double dt) {  
    velocity.add(LineareAlgebra.mult(dt, acceleration));  
    velocity.truncate(maxSpeed);  
    position.add(LineareAlgebra.mult(dt, velocity));  
}
```

$$\begin{aligned}\vec{a} \\ \overrightarrow{v_{t+\Delta t}} &= \overrightarrow{v_t} + \vec{a} \cdot \Delta t \\ \overrightarrow{p_{t+\Delta t}} &= \overrightarrow{p_t} + \overrightarrow{v_{t+\Delta t}} \cdot \Delta t\end{aligned}$$

[1] Millington I.: "Game Physics Engine Development: How to Build a Robust Commercial-Grade Physics Engine for your Game", 2nd ed., CRC Press, 2010

[2] Eberly D. H.: "Game Physics", 2nd ed., CRC Press, 2010



Aber woher bekommen wir jetzt die Beschleunigung?

Gesetze von Newton

Wenn wir ins Physikbuch schauen, finden wir zwei wichtige Sätze von Newton, die uns jetzt weiterhelfen: „*A force is a vector that causes an object with mass to accelerate.*“ und „*Force equals mass times acceleration.*“, was uns direkt zu folgender Formel führt:

$$\vec{a} = \frac{\vec{f}}{m}$$

Damit wollen wir unser Konzept verfeinern. Wenn wir die Masse eines Objekts für einen Moment mal außer Acht lassen und nur die Kraft betrachten, können wir eine Kraft als Beschleunigung speichern:

```
applyForce(Vektor3D force) {  
    acceleration = force;  
}
```

```
applyForce(wind);  
applyForce(gravitation);  
...
```

Wenn wir später aber mehrere Kräfte hinzufügen wollen, benötigen wir hier eine [Akkumulation](#) der einwirkenden Kräfte:

```
object.applyForce(wind);  
object.applyForce(gravitation);  
object.update();  
object.render();  
...
```

Gesetze von Newton

Ein zweiter Blick ins Physikbuch liefert uns eine gute Nachricht: „*Net Force equals mass times acceleration.*“. Dann müssen wir bei der Funktion **update** allerdings darauf achten, die Beschleunigung immer zu initialisieren, bzw. am Ende zu löschen:

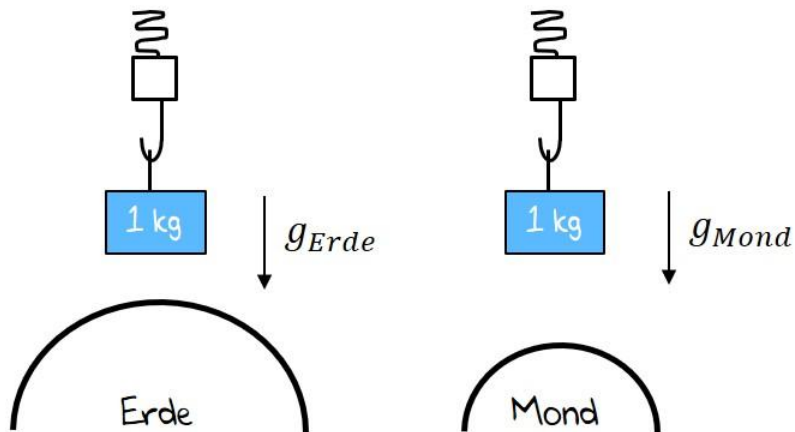
```
object.applyForce(wind);  
object.applyForce(gravitation);  
object.update();  
object.render();  
...
```

```
public void update() {  
    velocity.add(acceleration);  
    velocity.truncate(maxSpeed);  
    position.add(velocity);  
    acceleration.mult(0);  
}
```

Damit haben wir eine hervorragende Basis für weitere Untersuchungen und Erweiterungen geschaffen.

Masse versus Gewicht

Jetzt führen wir noch die **Masse** `mass` vom Datentyp `float` für ein Objekt ein. Die Masse eines Objekts ist nicht gleichbedeutend mit seinem Gewicht. Das **Gewicht** ergibt sich als **Produkt aus Masse und Gravitation**. So hat ein Objekt von einem Kilogramm auf der Erde ein um den Faktor 6 größeres Gewicht als auf dem Mond.



Starten wir den ersten direkten Versuch, der noch nicht sehr geschickt ist:

```
applyForce(Vektor3D force) {  
    force.div(mass);  
    acceleration = force;  
}
```

Was könnte hier schief gehen? Sollten wir versuchen eine Beschleunigungskraft **wind** zu erzeugen, um diese auf mehrere Akteure anzuwenden hätte das den Effekt, dass sich die Kraft nach jeder Anwendung deutlich verändert.

```
wind = ...  
obj1.applyForce(wind);  
obj2.applyForce(wind);  
...
```

Einführung in die Masse

Wir wollen die Kraft aber bei der Anwendung nicht verändern. Die Lösung ist ganz einfach und kann mit einer **lokalen Kopie** wie folgt aussehen:

```
applyForce(Vektor3D force) {  
    Vektor3D forceCopy = new Vektor3D(force);  
    forceCopy.div(mass);  
    acceleration.add(forceCopy);  
}
```

Unter Verwendung der nützlichen Klasse **LineareAlgebra**, können wir in diesem Beispiel auf einen der Parameter verzichten:

```
applyForce(Vektor3D force) {  
    acceleration.add(LineareAlgebra.div(force, mass));  
}
```

Damit wir ein sinnvolles **Steuerungsverhalten** erzeugen, müssen wir jetzt nur noch Kräfte erzeugen:

```
Vektor3D wind      = new Vektor3D(0.01, 0, 0);  
Vektor2D gravity = new Vektor2D(0, 0.1);  
...
```

Beispiel: Gravitationskraft

Galileo Galilei hat schon im Jahr 1589 mit Experimenten am schiefen Turm von Pisa festgestellt: [Die Gravitationsbeschleunigung ist nicht abhängig von der Masse](#). Wenn wir allerdings durch die Masse in **applyForce** immer teilen, können wir auf diese Weise die Gravitation geschickt unabhängig von der Masse machen:

```
Vektor2D gravity = new Vektor2D(0, 0.1 * mass);
```

So wird die Masse in der Methode **applyForce** später einfach weggelassen.

Beispiel: Reibungskraft zur Entschleunigung

Eine sehr einfache Kraft, die Simulationen ein wenig mehr Realismus verleihen kann, ist die **Reibung**. Ohne Reibung würde eine einmal in Gang gesetzte Bewegung bspw. mit dem Auto ohne weiteres Gasgeben unendlich fortgesetzt. Die Reibung ist eine Kraft, die der Einfachheit halber als Gegenkraft zur Geschwindigkeit gesehen werden kann:

$$\overrightarrow{f_{friction}} = -\mu N \cdot \text{normalize}(\vec{v})$$

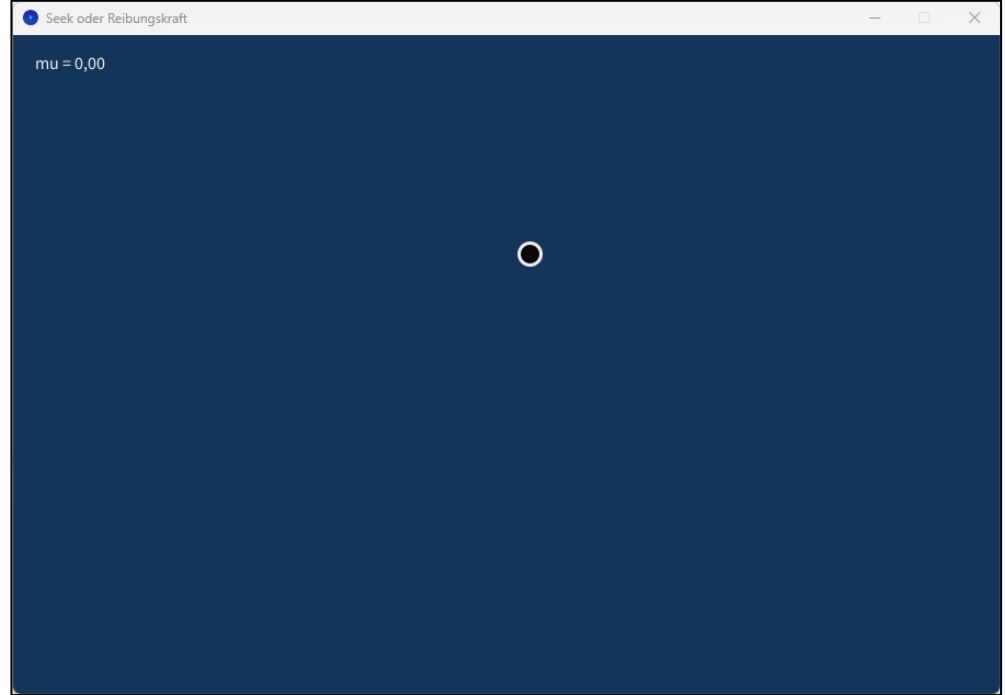
Die Erzeugung des Kraftvektors ist mit folgenden Codezeilen zu lösen, als Beispiel nehmen wir für $\mu = 0.01$:

```
Vektor3D friction = new Vektor3D(velocity);  
friction.mult(-1);  
friction.normalize();  
float mue = 0.01f;  
float N    = 1.0f;  
float frictionMagnitude = mue*N;  
friction.mult(frictionMagnitude);
```

Wir können jetzt durch den **Reibungskoeffizienten** μ die Eigenschaften und das Verhalten der Reibung bestimmen und die Kraft mit **applyForce** in unserer Simulation anwenden.

Reibungskraft mit unterschiedlichen Koeffizienten

Schauen wir uns hier ein kleines Beispiel in Processing an. Wir haben dafür das seek-Beispiel um eine Reibungskraft erweitert, wenn das Ziel ausgeblendet wird. Der Reibungskoeffizient ist dafür variabel auszuwählen.



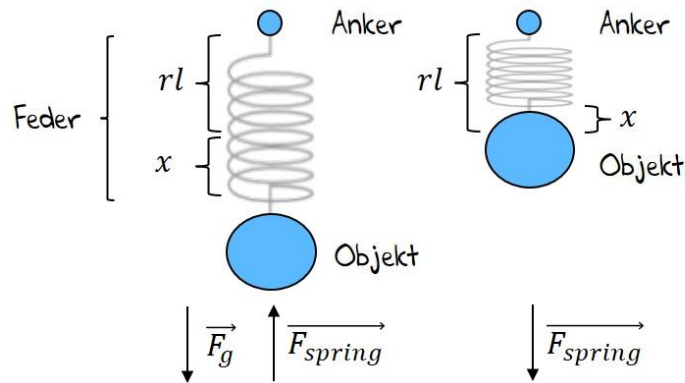
Beispiel: Einsatz der Federkraft

Das [Hooke'sche Gesetz](#) (nach Robert Hooke im Jahr 1678) besagt, dass die Dehnung linear von der wirkenden Kraft abhängt:

$$\overrightarrow{f_{spring}} = -k \cdot x$$

Wenn die Federkonstante **k**, der Federanker **anchor** und die Position des Federendes **location** bekannt sind, können wir die Kraft mit wenigen Codezeilen berechnen:

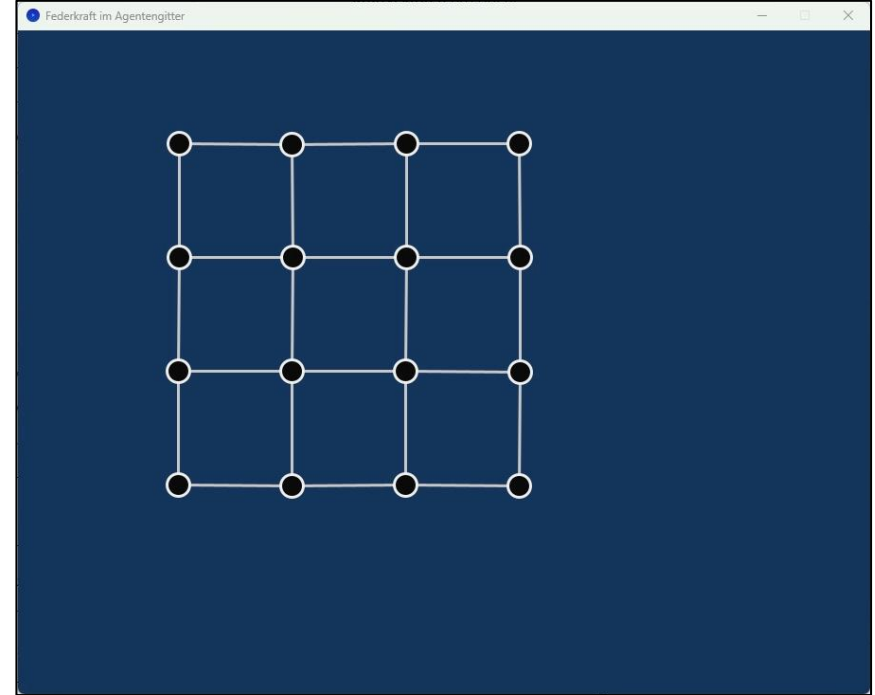
```
Vektor3D anchor;  
Vektor3D location;  
float rl;    // restLength  
float k = 0.1;  
Vektor3D force = LineareAlgebra.sub(location, anchor);  
float curLength = force.length;  
float x = rl - curLength;  
force.normalize();  
force.mult(-1 * k * x);
```



Die Kraft kann jetzt ebenfalls mit **applyForce** für viele Partikel direkt angewendet werden.

Federkraftbeispiel im Agentengitter

Schauen wir uns hier ein kleines Beispiel in Processing an. Hier haben die Agenten eine feste Formation, aus der sie per drag-and-drop gezogen werden können. Zwischen benachbarten Agenten sind Federn eingebaut.

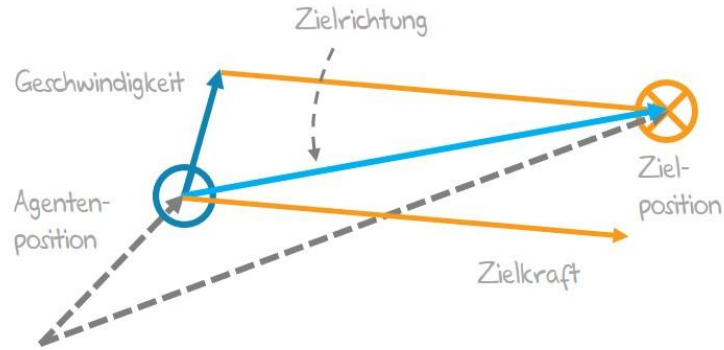




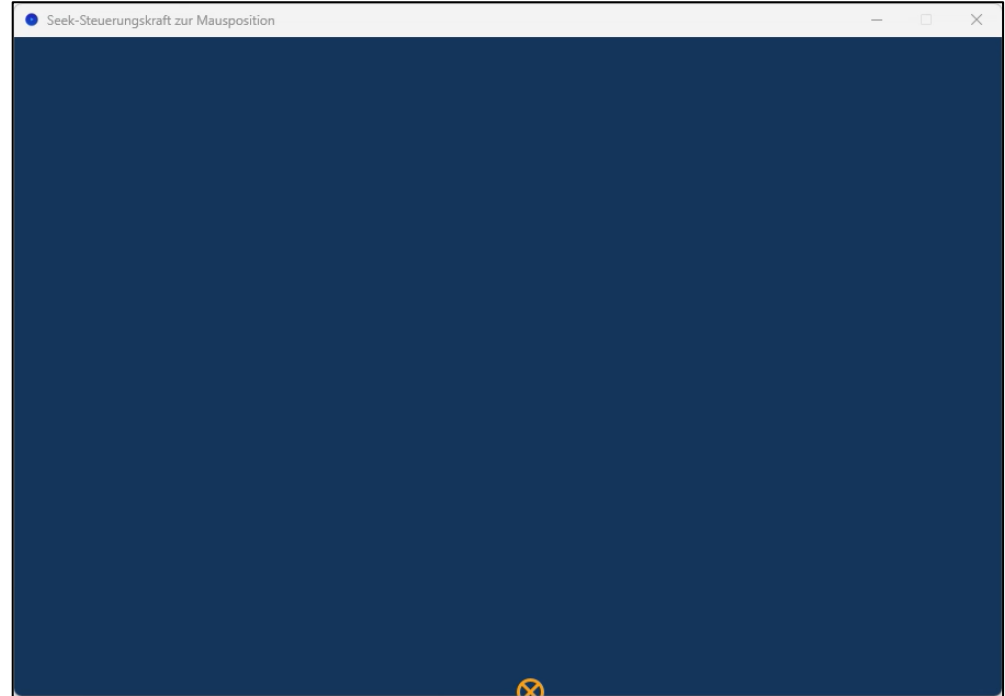
Kommen wir jetzt zu den Steuerungskräften

Steuerungskraft: Ziel folgen (seek)

Schauen wir uns die Wirkung einer Kraft auf einen Agenten am Beispiel „Ziel folgen“ an. Wenn wir einen Zielpunkt Ziel gegeben haben, dann gibt es eine Steuerungskraft, die direkt in die Richtung des Ziels zeigt.



Steuerungskraft „Ziel folgen“



Steuerungskraft: Ziel folgen (seek)

Wenn sich ein Agent A auf der Position $\vec{p}_A$ befindet und mit einer Geschwindigkeit $\vec{v}_A$ bewegt, dann soll eine **Steuerungskraft** $\vec{f}_{seek}$ den Agenten in Richtung einer Zielposition $\vec{p}_Z$ lenken. Dazu genügt es nicht, nur die reine Zielrichtung zu betrachten. Stattdessen berechnen wir zunächst die gewünschte Geschwindigkeit in Richtung des Ziels. Die eigentliche Steuerungskraft ergibt sich dann als Differenz zwischen dieser gewünschten Geschwindigkeit und der aktuellen Geschwindigkeit des Agenten.

```
public Vektor2D forceSeek(Vektor2D currentPosition,
                          Vektor2D currentVelocity,
                          Vektor2D target,
                          double maxSpeed,
                          double maxForce) {
    Vektor2D desired = LineareAlgebra.sub(target, currentPosition);
    if (desired.length() > 0.0001) {
        desired.normalize();
        desired.mult(maxSpeed);
    }
    Vektor2D steering = LineareAlgebra.sub(desired, currentVelocity);
    steering.truncate(maxForce);
    return steering;
}
```

$$\vec{desired} = \text{normalize}(\vec{p}_Z - \vec{p}_A) \cdot \vec{v}_{max}$$

$$\vec{f}_{seek} = \vec{desired} - \vec{v}_A$$

Optional können wir die Kraft noch auf eine maximale Stärke f_{max} beschränken:

$$\vec{f}_{seek} = \text{limit}(\vec{f}_{seek}, f_{max})$$

Seek ist nicht „Richtung zum Ziel“, sondern „Differenz zwischen gewünschter und aktueller Geschwindigkeit“.

Steuerungskraft: Fliehen (flee)

Ist das Ziel bspw. ein Feind und wir wollen eine Kraft $\overrightarrow{f_{flee}}$ genau in die entgegengesetzte Richtung konstruieren, müssen wir die Richtung einfach nur umdrehen.

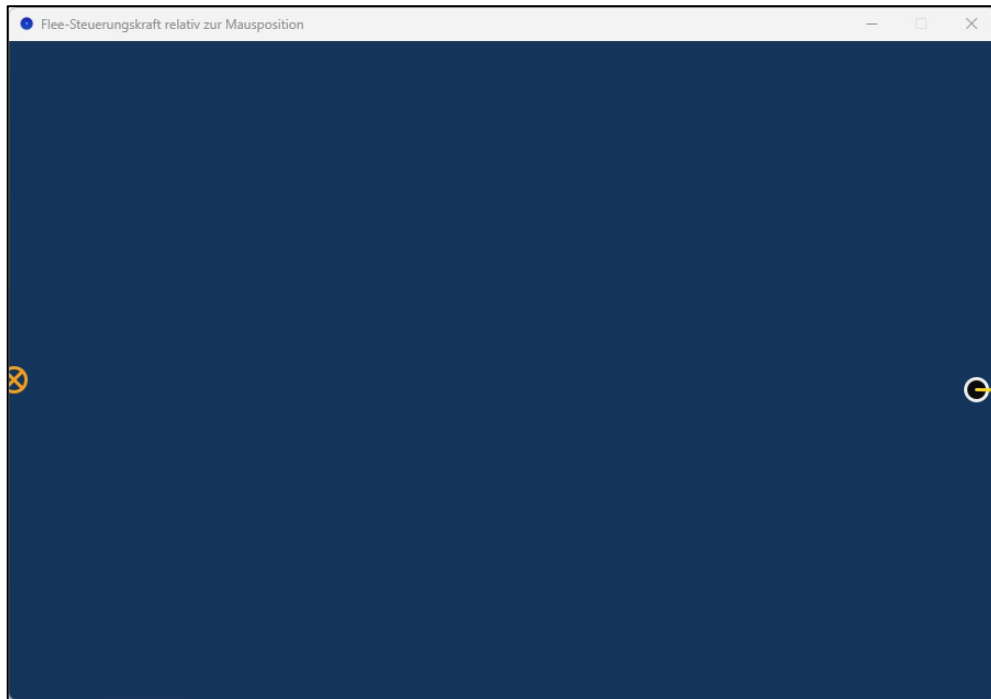
```
public Vektor2D forceFlee(Vektor2D currentPosition,
                        Vektor2D currentVelocity,
                        Vektor2D target,
                        double maxSpeed,
                        double maxForce) {
    return forceSeek(currentPosition, currentVelocity, target, maxSpeed, maxForce).mult(-1);
}
```

$$\overrightarrow{f_{flee}} = -\overrightarrow{f_{seek}}$$

Steuerungskraft: Fliehen mit harter Randbedingung

Schauen wir uns die Wirkung einer Kraft auf einen Agenten am Beispiel „Fliehen“ an. Hier verwenden wir eine harte Randbedingung, die den Agenten direkt abbremst.

```
void update() {  
    velocity.add(acceleration);  
    velocity.limit(maxSpeed);  
    position.add(velocity);  
    // Randbegrenzung  
    if (position.x < radius) {  
        position.x = radius;  
        if (velocity.x < 0) velocity.x = 0;  
    }  
    if (position.x > width - radius) {  
        position.x = width - radius;  
        if (velocity.x > 0) velocity.x = 0;  
    }  
    if (position.y < radius) {  
        position.y = radius;  
        if (velocity.y < 0) velocity.y = 0;  
    }  
    if (position.y > height - radius) {  
        position.y = height - radius;  
        if (velocity.y > 0) velocity.y = 0;  
    }  
    acceleration.mult(0);  
}
```



Steuerungskraft: Fliehen mit Boundary-Force

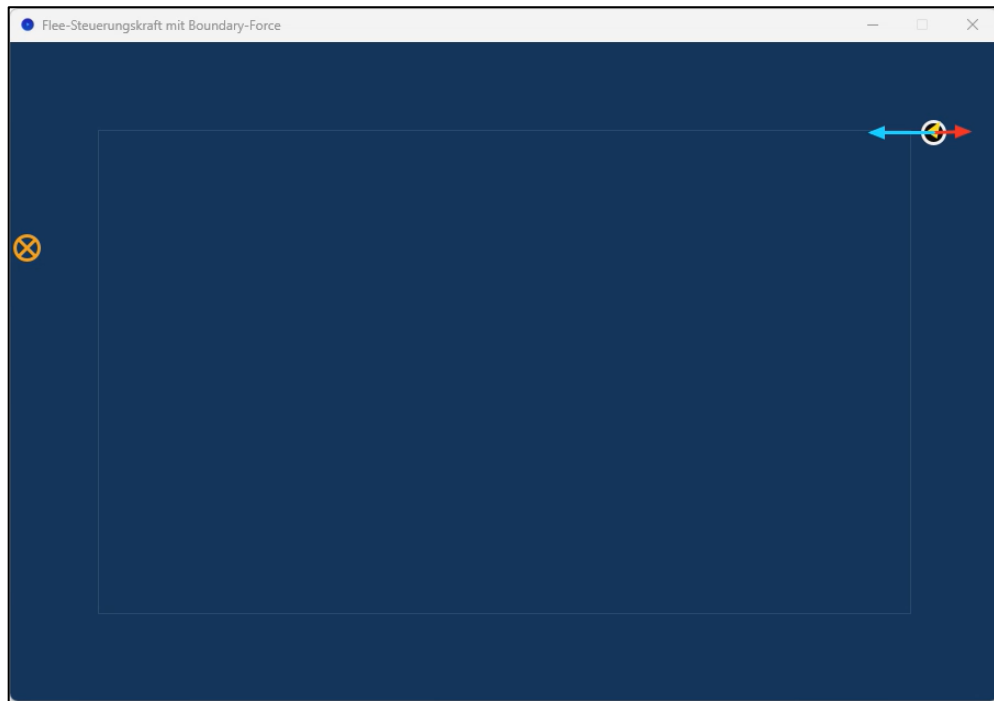
Steuerungskräfte können uns auch hier helfen. Wenn der Agent einem Rand zu nahe kommt, erzeugen wir eine zusätzliche Kraft (**Boundary-Force**), die ihn wieder ins Innere lenkt. Das liefert ein sehr viel besseres Verhalten.

$$\overrightarrow{desired_{boundary}} = \begin{cases} (v_{max}, v_y) & \text{links} \\ (-v_{max}, v_y) & \text{rechts} \\ (v_x, v_{max}) & \text{oben} \\ (v_x, -v_{max}) & \text{unten} \end{cases}$$

$$\overrightarrow{f_{boundary}} = \overrightarrow{desired_{boundary}} - \overrightarrow{v_A}$$

Optional können wir die Kraft noch auf eine maximale Stärke f_{max} beschränken:

$$\overrightarrow{f_{boundary}} = \text{limit}(\overrightarrow{f_{boundary}}, f_{max})$$



Steuerungskraft: Hindernisumgehung (obstacle avoidance)

In einer realistischeren Simulationsumgebung müssen Agenten zusätzlich auf **Hindernisse im Raum** reagieren. Die Grundidee der Hindernisvermeidung lautet: Der Agent blickt in seine aktuelle Bewegungsrichtung. Er prüft, ob auf seiner voraussichtlichen Bahn ein Hindernis liegt. Falls eine Kollision droht, wird eine Ausweichkraft berechnet. Damit wird Hindernisvermeidung nicht als globale Planung, sondern als **lokale Reaktion** modelliert.

```
public Vektor2D obstacleAvoidance(Vektor2D position,
                                   Vektor2D velocity,
                                   Vektor2D obstaclePos,
                                   double obstacleRadius,
                                   double lookAhead,
                                   double maxSpeed,
                                   double maxForce) {

    if (velocity.length() < 0.0001)
        return new Vektor2D(0, 0);

    Vektor2D ahead = new Vektor2D(velocity);
    ahead.normalize();
    ahead.mult(lookAhead);
    ahead.add(position);

    double dist = LineareAlgebra.euklDistanz(ahead,
                                              obstaclePos);
```

```
    if (dist < obstacleRadius) {
        Vektor2D desired = LineareAlgebra.sub(ahead,
                                              obstaclePos);

        desired.normalize();
        desired.mult(maxSpeed);

        Vektor2D steering = LineareAlgebra.sub(desired,
                                              velocity);

        steering.truncate(maxForce);
        return steering;
    }

    return new Vektor2D(0, 0);
}
```

Steuerungskraft: Hindernisumgehung (obstacle avoidance)

Für die aktuelle Position $\vec{p}_A$ und die Geschwindigkeit $\vec{v}_A$ bestimmen wir zunächst einen **Blickpunkt** vor dem Agenten $\vec{ahead} = \vec{p}_A + \text{normalize}(\vec{v}_A) \cdot d$ mit einer **Vorhersagedistanz** d .

Liegt in der Nähe dieses Punkts ein Hinderniszentrum $\vec{p}_H$, dann wählen wir eine Ausweichrichtung vom Hindernis weg:

$$\vec{desired}_{avoid} = \text{normalize}(\vec{ahead} - \vec{p}_H) \cdot \vec{v}_{max}$$

$$\vec{f}_{avoid} = \vec{desired}_{avoid} - \vec{v}_A$$

Optional können wir die Kraft noch auf eine maximale Stärke f_{max} beschränken:

$$\vec{f}_{avoid} = \text{limit}(\vec{f}_{avoid}, f_{max})$$

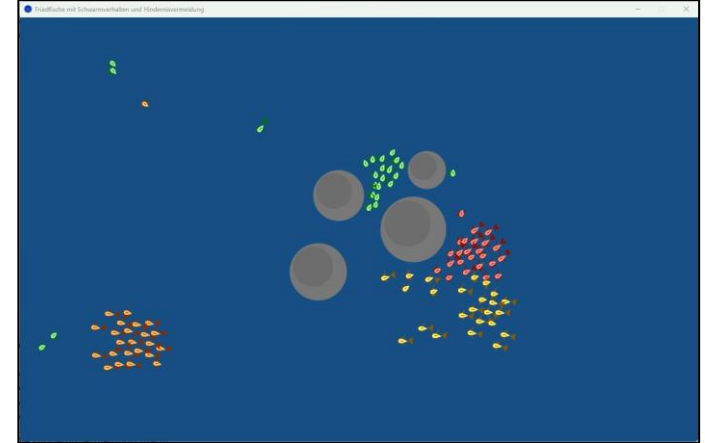
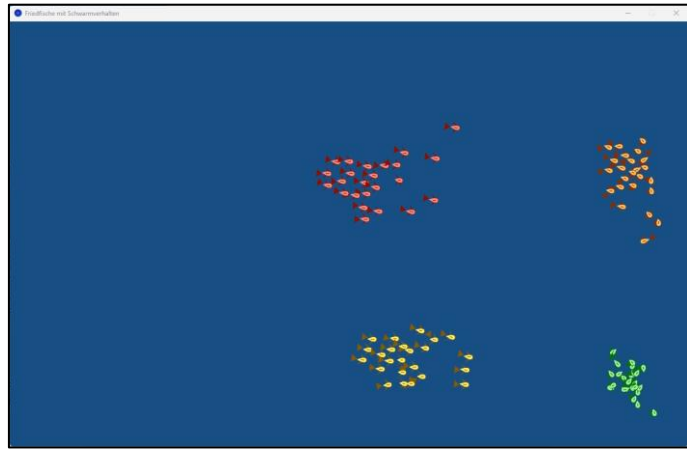




Wie funktioniert eigentlich die Bewegung
mehrerer Agenten gleichzeitig?

Kollektive Intelligenz

Die Untersuchung von **kollektiver Intelligenz** ist gerade in den Bereichen Biologie, Soziologie, Netzwerktheorie und der Künstlichen Intelligenz interessant. Hier stellt sich die Frage, welche Mechanismen bei Individuen verankert sind, um in der Gemeinschaft komplexe Aufgaben zu lösen.



Das **Schwarmverhalten** von Vögeln, Fischen usw. lässt sich auf sehr einfache Weise simulieren. In verschiedenen **Spiele** und **Filmen** (Der König der Löwen 1994, Batmans Rückkehr 1989, Herr der Ringe 2001) wird dieses Schwarmverhalten eingesetzt.

`spielumgebung/Schwarmverhalten_Fische`

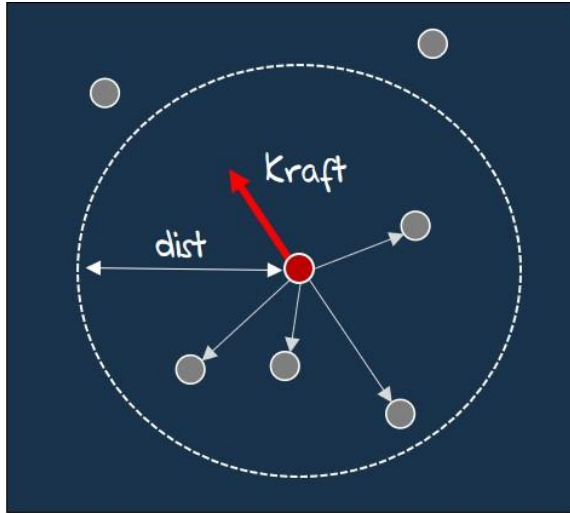
`spielumgebung/Schwarmverhalten_Fische_ObstacleAvoidance`



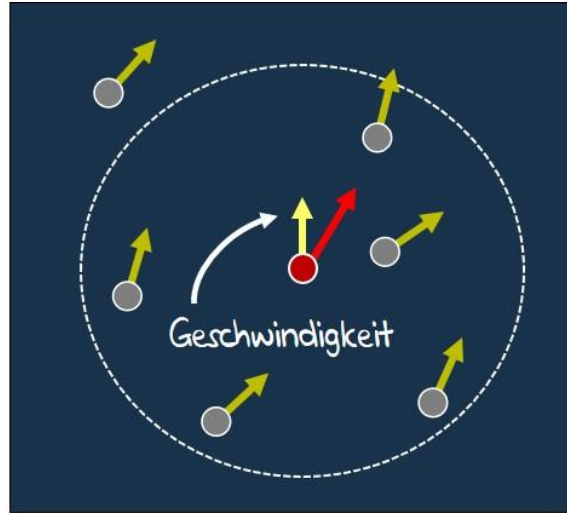
Steuerungskräfte für ein **Schwarmverhalten**

Die drei Kräfte der Flocking-Boids

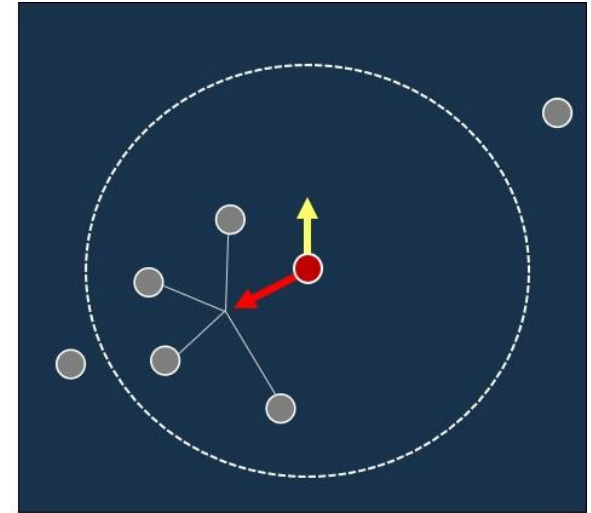
Mit den **Flocking-Boids** wurde 1986 von Craig Reynolds eine Methode basierend auf drei einfachen Regeln eingeführt, mit der sich interessante und realistisch wirkende **Schwarmbewegungen** von Vögeln oder Fischen auf sehr einfache Weise simulieren lassen [1].



Separation
Individueller Freiraum



Alignment
Gleiche Richtung der Gruppe



Cohesion
Zusammenhalt zur Gruppe

[1] Reynolds C. W.: "Flocks, Herds, and Schools: A Distributed Behavioral Model", ACM SIGGRAPH Computer Graphics, 1987
<https://www.red3d.com/cwr/boids/>

Separationsregel

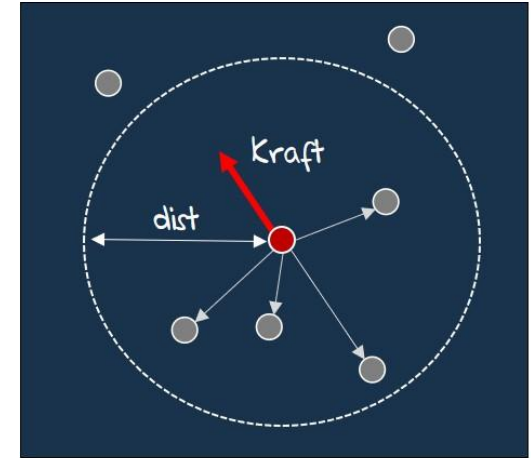
Wenn innerhalb einer Entfernung **dist** andere Objekte vorhanden sind, bewegt sich das Individuum in die **Richtung mit dem größten Freiheitsgrad**. Um den Separationsvektor zu erhalten, müssen wir lediglich alle Objekte identifizieren, die innerhalb der Umgebung **dist** vorhanden sind. Wenn wir für jedes dieser Objekte den Vektor in unsere Richtung erzeugen, aufsummieren und am Ende normalisieren, erhalten wir die Richtung $\overrightarrow{f_{separation}}$ des größten Freiheitsgrades:

```
public Vektor2D separation(Agent me, float dist) {
    Vektor2D desired = new Vektor2D(0, 0);
    int count = 0;

    for (int i = 0; i < me.objektManager.getAgentSize(); i++) {
        if (me.id == i) continue;

        BasisObjekt bObj = me.objektManager.getAgent(i);
        if (bObj instanceof Agent) {
            Agent other = (Agent)bObj;
            double d = LineareAlgebra.euklDistanz(me.position, other.position);
            if (d > 0 && d < dist) {
                Vektor2D diff = LineareAlgebra.sub(me.position, other.position);
                diff.normalize(); diff.div(d); desired.add(diff); count++;
            }
        }
    }

    if (count == 0) return new Vektor2D(0, 0);
    desired.div(count); desired.normalize(); desired.mult(me.maxSpeed);
    Vektor2D steeringForce = LineareAlgebra.sub(desired, me.velocity);
    steeringForce.truncate(me.maxForce);
    return steeringForce;
}
```



Separation
Individueller Freiraum

Angleichungs- oder Alignmentregel

Die Bewegungsrichtung eines Individuums entspricht der **mittleren Richtung aller umliegenden**. Auch hier sammeln wir die Richtungen ein, summieren sie auf und normalisieren die Richtung am Ende zu $\overrightarrow{f_{alignment}}$:

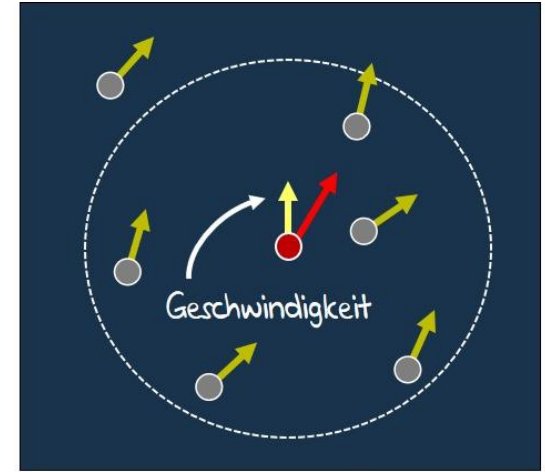
```
public Vektor2D alignment(Agent me, float dist) {
    Vektor2D desired = new Vektor2D(0, 0);
    int count = 0;

    for (int i = 0; i < me.objektManager.getAgentSize(); i++) {
        if (me.id == i) continue;

        BasisObjekt bObj = me.objektManager.getAgent(i);
        if (bObj instanceof Agent) {
            Agent other = (Agent)bObj;
            if (LineareAlgebra.euclidDistanz(me.position, other.position) < dist) {
                desired.add(other.velocity);
                count++;
            }
        }
    }

    if (count == 0) return new Vektor2D(0, 0);
    desired.div(count); desired.normalize(); desired.mult(me.maxSpeed);

    Vektor2D steeringForce = LineareAlgebra.sub(desired, me.velocity);
    steeringForce.truncate(me.maxForce);
    return steeringForce;
}
```



Alignment
Gleiche Richtung der Gruppe

Zusammenhalt- oder Cohesionsregel

Die Bewegungsrichtung entspricht auch der **mittleren Position der anderen Gruppenmitglieder**. Diese Methode ist der Alignmentmethode sehr ähnlich. Hier wird lediglich die mittlere Position durch Aufsummieren ermittelt. Am Ende ergibt sich die normalisierte Richtung $\overrightarrow{f_{cohesion}}$ zu dieser zentralen Position:

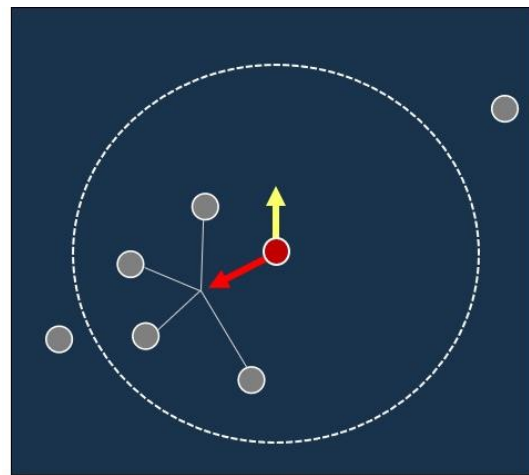
```
public Vektor2D cohesion(Agent me, float dist) {
    Vektor2D center = new Vektor2D(0, 0);
    int count = 0;

    for (int i = 0; i < me.objektManager.getAgentSize(); i++) {
        if (me.id == i) continue;

        BasisObjekt bObj = me.objektManager.getAgent(i);
        if (bObj instanceof Agent) {
            Agent other = (Agent)bObj;
            if (LineareAlgebra.euklDistanz(me.position, other.position) < dist) {
                center.add(other.position);
                count++;
            }
        }
    }

    if (count == 0) return new Vektor2D(0, 0);
    center.div(count);

    return forceSeek(me.position, me.velocity, center, me.maxSpeed, me.maxForce);
}
```



Cohesion
Zusammenhalt zur Gruppe

Zusammenfassen der Kräfte

Wir haben drei Kräftevektoren ermittelt und müssen diese jetzt noch zu einer Kraft zusammenbringen. Dazu werden die **Kräfte gewichtet summiert** zu (szenenabhängig zu tunen):

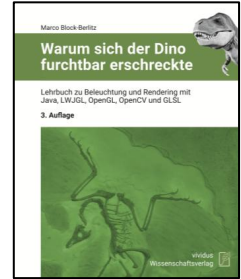
$$\vec{f}_{\text{schwarm}} = 1.0 \cdot \vec{f}_{\text{separation}} + 0.12 \cdot \vec{f}_{\text{alignment}} + 0.01 \cdot \vec{f}_{\text{cohesion}}$$

Die Separation, also die individuelle Freiheit, stellt den wichtigsten Anteil dar. Danach kommen typischerweise Alignment und Cohesion. Angewendet auf die Agenten könnte das bspw. so aussehen:

```
public void update(double dt) {  
    Vektor2D separationForce = steering.separation(agent, 10);  
    separationForce.mult(1.0);  
    agent.applyForce(separationForce);  
  
    Vektor2D alignmentForce = steering.alignment(agent, 15);  
    alignmentForce.mult(0.12);  
    agent.applyForce(alignmentForce);  
  
    Vektor2D cohesionForce = steering.cohesion(agent, 15);  
    cohesionForce.mult(0.01);  
    agent.applyForce(cohesionForce);  
  
    agent.update(dt);  
}
```

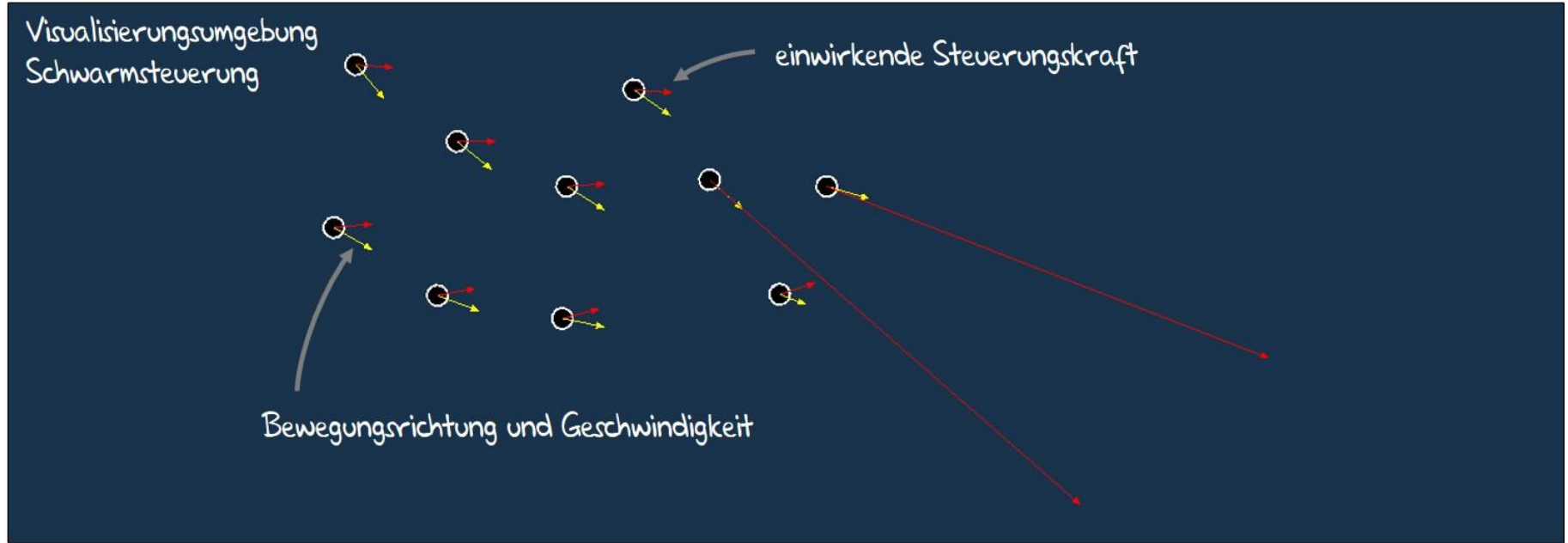
Übersicht zu der Programmsammlung

- version00** das minimale Eingangsbeispiel zum perfekten Flummi
- version01** Basisobjekt und Kreis kommen in der WeltDesEinsamenKreises zusammen
- version02** BewegendesObjekt wird von BasisObjekt abgeleitet und Flummi als erstes Beispiel erzeugt
- version03** Erweiterung des Beispiels um den Objektmanager, damit die Verwaltung mehrerer Objekte möglich ist
- version04** Kapselung des Verhaltens als Interface und damit Komposition
- version05** die Umstellung der in Kapitel 4 vorgestellten Vektorklassen erfolgt in der Simulationsumgebung
- version06** hier wird die erste Steuerungskraft mittels Zufall implementiert und eingesetzt
- version07** wenn wir ausschließlich die Mausposition als Steuerungskraft einsetzen, sehen wir, dass nach kurzer Zeit alle Objekte zu einer Position streben
- version08** wenn wir zur Mausposition noch die Separation bringen, sieht das Schwarmverhalten schon ziemlich interessant aus
- version09** jetzt kommt noch die Steuerungskraft Alignment hinzu
- version10** die neue Steuerungskraft Cohesion wird eingebaut
- version11** ein kleines Partikelsystem, bei dem verschiedene Kräfte wirken
- version12** damit die einzelnen Steuerungskräfte entwickelt, untersucht und ausprobiert werden können, gibt es dieses Projekt
- version13** viele der Steuerungskräfte wirken erst, wenn es mehrere Objekte gibt, daher ist das eine interessante Spielwiese mit zahlreichen Visualisierungen für Schwärme



Schwarmsimulation

Hier sehen wir 10 Agenten in Aktion. Die beiden relevanten Vektoren einwirkende Steuerungskraft (Beschleunigung) und Bewegungsrichtung werden angezeigt.



Visualisierungskomponenten in POGGL

In der Klasse **POGL** finden sich nützliche **Visualisierungskomponenten** für eine Schwarmsimulation:

```
public static void renderSwarmObjectWithForces(float x, float y, int radius, Vektor2D velocity, Vektor2D acceleration) {
    glLoadIdentity();
    glTranslated(x, y, 0);
    glColor4f(1, 1, 1, 1);
    renderKreis(0, 0, 5, radius);
    glColor4f(0, 0, 0, 1);
    renderKreis(0, 0, 5, radius-2);

    int off = radius + 1 + (int)(velocity.length()/5);
    double winkel = LineareAlgebra.angleDegree(velocity, new Vektor2D(1,0));
    if (velocity.y<0)
        winkel = 180 + (180-winkel);

    glColor4f(1, 1, 0, 1);
    renderPfeil(x, y, off, (float)winkel, 15);
    off = radius + 1 + (int)(acceleration.length()/10);
    winkel = LineareAlgebra.angleDegree(acceleration, new Vektor2D(1,0));
    if (acceleration.y<0)
        winkel = 180 + (180-winkel);
    glColor4f(1, 0, 0, 1);
    renderPfeil(x, y, off, (float)winkel, 15);
}
```

```
public static void renderPfeil(float x, float y,
                               int off, float winkel,
                               int size) {

    glLoadIdentity();
    glTranslated(x, y, 0);
    glRotatef(winkel, 0, 0, 1);
    glTranslated(off, 0, 0);
    glScaled(size, size, size);

    glBegin(GL_LINES);
        glVertex3d( 0f, 0f, 0);
        glVertex3d(-off/15., 0, 0);
    glEnd();

    glBegin(GL_TRIANGLES);
        glVertex3d( 0f,  .2f, 0);
        glVertex3d( 0f, -.2f, 0);
        glVertex3d( .5f,  0f, 0);
    glEnd();
}
```

Action Selection, Steering und Locomotion im Fischschwarm

Am Beispiel des Fischschwarms lassen sich die drei Ebenen autonomer Agenten noch einmal konkret unterscheiden (Action Selection, Steering, Locomotion):

Action Selection

Welche grundlegende Verhaltensweise ist gerade gewünscht?

- im Schwarm bleiben
- Abstand zu Nachbarn halten
- Hindernisse vermeiden
- im Becken bleiben

Steering

Wie werden aus diesen Zielen konkrete Steuerungskräfte?

- Separation
- Alignment
- Cohesion
- Boundary Force
- Obstacle Avoidance

Locomotion

Wie wird aus der Kraft eine sichtbare Bewegung?

- Beschleunigung aus Kräften berechnen
- Geschwindigkeit aktualisieren
- Position integrieren
- Fischgrafik in Bewegungsrichtung darstellen

Der Fischschwarm ist kein einzelner Algorithmus, sondern das Zusammenspiel von Zielwahl, Steuerung und Bewegungsausführung.



Wie könnte es an dieser Stelle **weitergehen**?

Einfache Spielumgebung als Untersuchungslabor

Mit einer sehr einfachen Umsetzung einer Spielwelt lassen sich bereits einige wichtige **Aspekte der Computergrafik** im Bereich visueller Simulationen erläutern.

Typische Elemente

- Debug-Umgebung
- Statische, dynamische Objekte
- Kapselung von Verhalten
- Objektmanager
- Steuerungsverhalten
- Hinderniserkennung
- Schwarmverhalten
- Wassereffekt
- ...



Refraktion und PixelArt

Mit einfachen Tricks lassen sich z. B. durch **Refraktionseffekte** klassische PixelArt-Visualisierungen erzeugen.

Wer kennt noch das legendäre Point-and-Click-Adventure **Monkey Island** [1]?



spielumgebung/KorallenRiffDebug



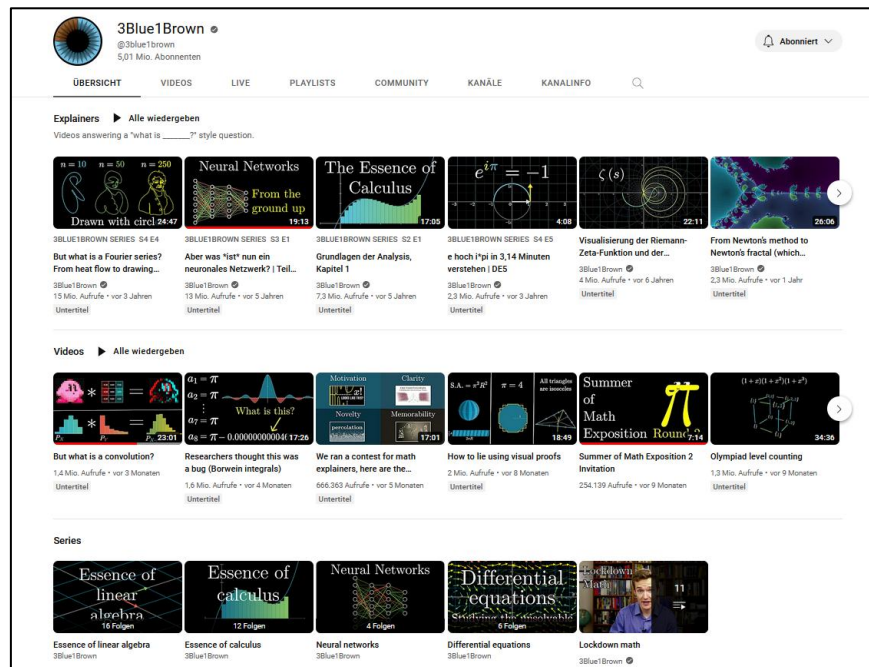
[1] MONKEY ISLAND 1 [HD] - The Secret of Monkey Island ★ Monkey Island Longplay: <https://youtu.be/mtRxcBZJz0I?si=R4CgrrAbVBdgoox3>



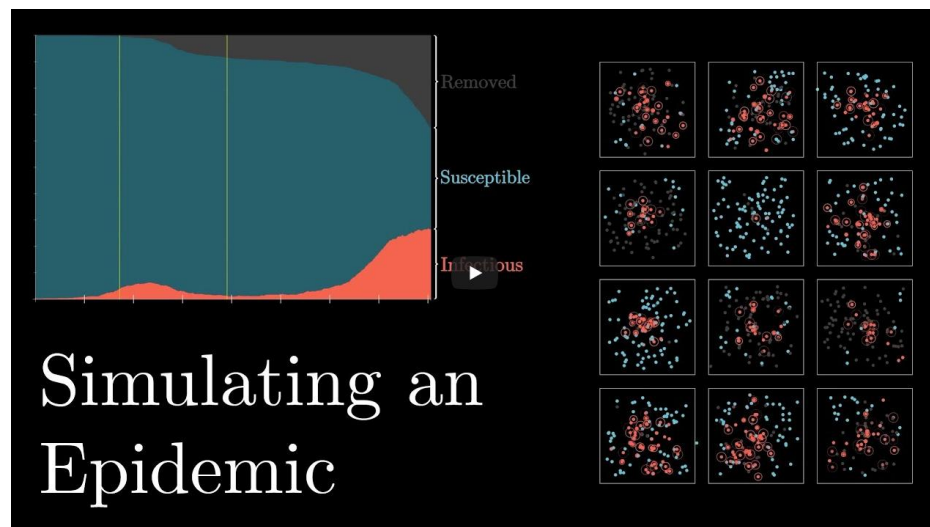
Was können wir jetzt damit anfangen,
wenn wir **keine Spiele** machen wollen ...?

Empfehlung: 3Blue1Brown

Wie sagte Marcel Reich-Ranicki: „Das Fernsehen macht die Klugen klüger und die Dummen dümmer!“. Viele von Euch zappen sicherlich manchmal durch YouTube. Hier lassen sich auch sehr anschauliche wissenschaftliche Videos zu komplexen Themen z. B. im YouTube-Kanal von [3Blue1Brown](#) finden.

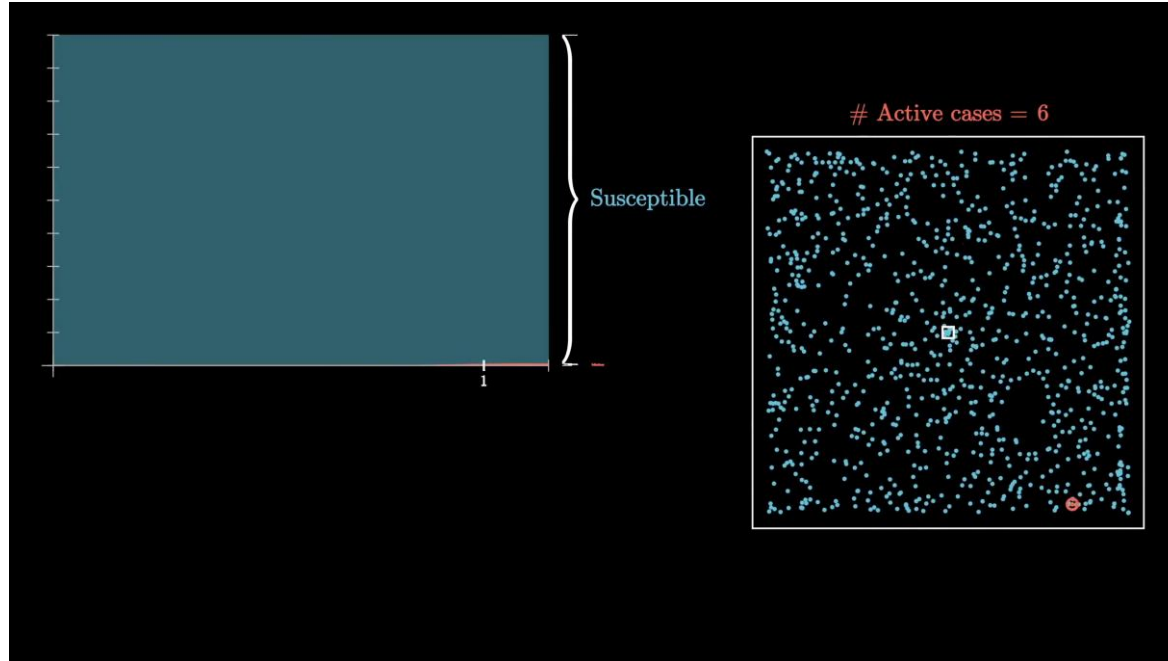


The screenshot shows the 3Blue1Brown YouTube channel page. The channel name is 3Blue1Brown, with 5,01 Mio. Abonnenten. The page is divided into sections: Explainers, Videos, and Series. The Explainers section features a grid of video thumbnails with titles like 'But what is a Fourier series?', 'Neural Networks', 'The Essence of Calculus', 'e hoch i*pi in 3,14 Minuten verstehen', 'Visualisierung der Riemann-Zeta-Funktion', and 'From Newton's method to Newton's fractal'. The Videos section shows thumbnails for 'But what is a convolution?', 'Researchers thought this was a bug (Borwein integrals)', 'We ran a contest for math explainers', 'How to lie using visual proofs', 'Summer of Math Exposition Round 7', and 'Olympiad level counting'. The Series section includes 'Essence of linear algebra', 'Essence of calculus', 'Neural Networks', 'Differential equations', and 'Lockdown math'.



Simulating an Epidemic

Zu Zeiten der Corona-Pandemie gab es eine Reihe interessanter Simulationen [1] zur Erläuterung von Ansteckungsgrad, Verbreitung, Social-Distancing, Isolation usw. Wir wurden in den letzten Jahren zu Pseudo-Epidemiologen ausgebildet ...



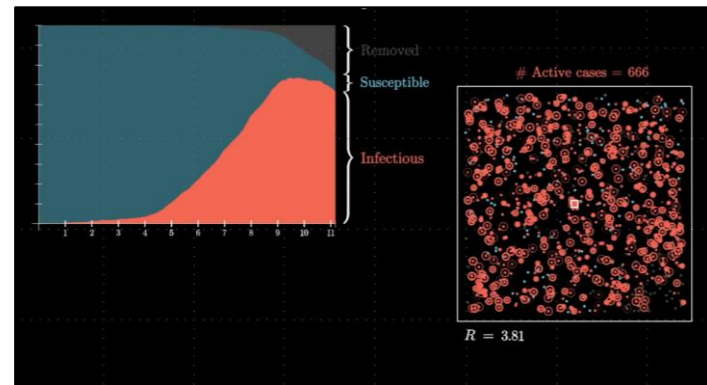
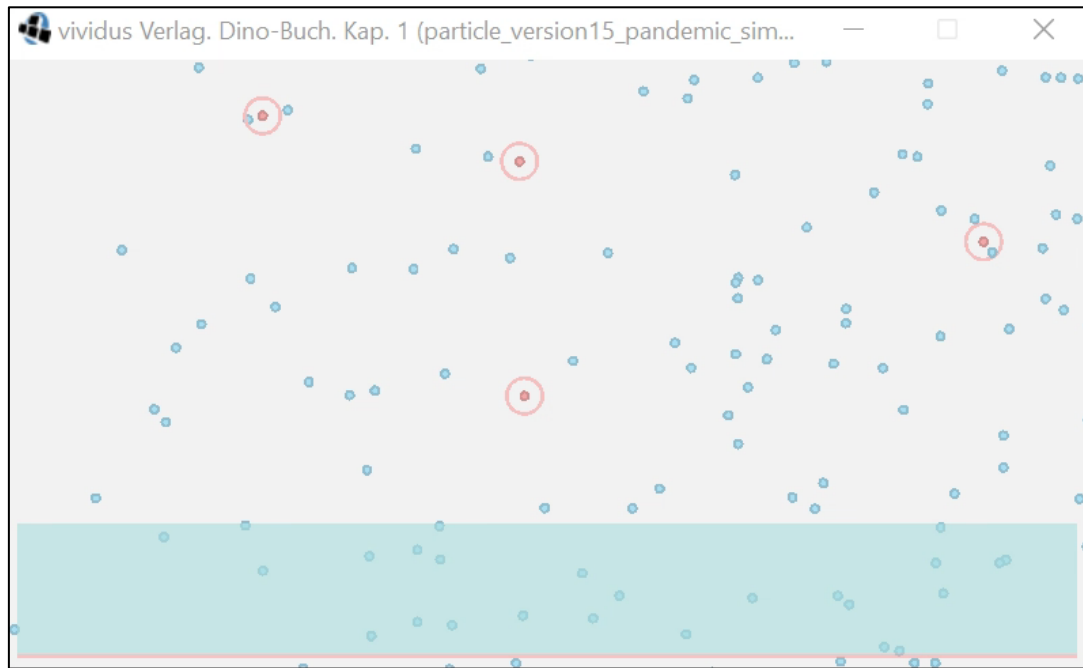
[1] <https://www.3blue1brown.com/lessons/epidemic-simulations>



Können wir so eine **Simulation** auch
in unserer einfachen Spielumgebung umsetzen?

COVID-19-Simulation

Ich habe Euch eine kleine Erweiterung der Simulationsumgebung gebastelt (hat keine 2h gedauert), bei der die Agenten einen Virus verbreiten können. **Blau** sind die Gesunden, **hell-rot** sind die Infizierten, **dunkel-rot** die Infektiösen und **grau** die Genesenen.



Kollektive Intelligenz und Schwarmverhalten

Key Takeaways



- Verhalten kann Kräfte erzeugen
- Ein Agent braucht mindestens Position, Geschwindigkeit und Beschleunigung
- Kräfte werden pro Simulationsschritt akkumuliert und danach zurückgesetzt
- Zeitintegration mit Δt macht Bewegung unabhängig von der Update-Frequenz
- Einfache Kraftmodelle wie Gravitation, Reibung oder Federkraft reichen für viele visuelle Simulationen bereits aus
- Lokale Regeln erzeugen globale Muster
- Schwarmverhalten entsteht ohne zentrale Steuerung
- Steering-Kräfte lassen sich kombinieren

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

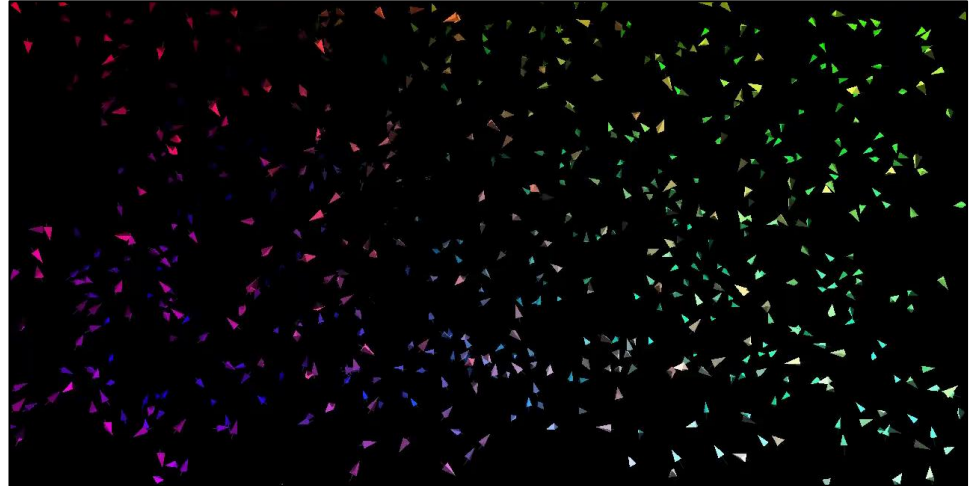
- (1) Beschreiben Sie allgemein, was unter dem Konzept Steuerungsverhalten verstanden wird.
- (2) Worin besteht der Unterschied zwischen den Steuerungskräften seek und flee?
- (3) Erweitern Sie die Steuerungskraft seek zu arrive, damit wir direkt am Ziel langsam ankommen und dort stehen bleiben.
- (4) Wieso ist Schwarmverhalten ein Beispiel für Emergenz?
- (5) Warum ist es sinnvoll, die beiden Methoden **render** (Objekt wird gezeichnet) und **update** (Objektposition, -orientierung, ... wird angepasst) zu trennen?
- (6) Welche Rolle spielt das Strategy-Pattern bei der Integration von Verhalten? Welchen Vorteil erhalten wir bei der Verwendung?
- (7) Welche Rolle spielen Separation, Alignment und Cohesion im Zusammenspiel?



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

- (1) **Aufgabe 1)** Entwickeln Sie ein eigenes Konzept für eine andere Art von Schwarm oder kollektiver Bewegung. Ihre Aufgabe besteht darin, ein neues Szenario zu modellieren, zum Beispiel: Vogelschwarm, Insektenschwarm, Drohnenschwarm, Menschenmenge, Fantasiewesen, Partikelschwarm mit eigenem Verhalten. Wichtig ist, dass Sie das Verhalten nicht sofort implementieren, sondern zunächst als Agentensystem analysieren und strukturieren (Action Selection, Steering, Locomotion).



Zur Inspiration



Vielen Dank für die Aufmerksamkeit